Demixing Complex Human Behaviour

T. Quendera

0.1 Notes on versions

0.1.1 v0.1 - 28/12/2023

Changelog:
- This version is an early draft for the methods and results for the hexxed chapter 4.1.

Useful feedback:
- Overall tone.
- On the methods section.

Author Contributions and Financial Support

Acknowledgements

Abstract

an abstract

Thesis Overview

Contents

List of Figures

Glossary

4.1.0.1 The rise of AI From arcade classic Space Invaders to one of the most complex games ever played in Go, there has been a noticeable shift in prowess between human players and artificial intelligence. Today, machines reign supreme in tasks we deemed human exclusive only a few decades ago. This shift can be primarily attributed to advancements in neural network algorithms. In 1997, Gary Kasparov - the best of human chess players - was defeated using the brute-force algorithm Deep Blue [Hsu, 2002]. Albeit incredibly powerful, this algorithm depended on very specific hardware and software configurations to be good at the game. Deep Blue was exclusively good at the game of chess. It contained a tree of millions of possible chess plays, a database of the most common openings and endgames and the aid of human engineers to steer it in the right direction. The definitive domination over humans in this type of tasks only came with the dawn of Machine Learning and neural networks. Neural networks are a type of algorithms inspired by the neuron-based, layered way in which biological brains compute problems. Due to their architecture, they are capable of learning from data. During training, they adjust their internal parameters

(weights and biases) to adjust to the problem at hand. This training involves using large amounts of data to improve the network's performance at a given task, whether it's recognizing images, understanding language, or playing complex games. These algorithms are capable of generalizing: they adjust to both the task at hand and, even with incomplete data (contrarily to brute-force algorithms), are able to infer what to do in a never before seen problem of the same kind.

4.1.0.2 Investigating complex skill learning Despite this, one aspect where humans continue to hold a significant edge is in the speed at which we acquire and refine our skills [Tsividis et al., 2017] [Lake et al., 2017]. Lake and colleagues pose that the differences in learning may be due to the different learning apparatuses. Complex skill learning, can be formalized as a journey through a vast space of potential strategies. It is by traversing this complex strategy space that agents eventually land on critical points that lead to task performance - i.e. they learn. While designed as reward-based gradient descent for artificial agents [Mnih et al., 2015], the exact nature of how humans navigate this space remains uncovered. One position is that it follows a similar process. Alternatively, some pose this process is influenced by rational inference and priors [Tsividis et al., 2017] or other processes [Martin-Maroto and de Polavieja, 2018]. Given the wealth of theories, a significant challenge in this field of research remains that games serve as excellent benchmarks for assessing intelligent search strategies and decision-making processes yet, they are not inherently designed to reverse-engineer complex skill learning. If we are to draw parallelism between different types of learning in different agents, we ought to investigate tasks that are designed to investigate underlying cognitive processes - i.e. learning.

4.1.0.3 Not too simple, not too complex On the opposite end of the spectrum of complexity, the tools of psychophysics have been instrumental in many of the great findings in cognitive psychology. The field of psychophysics examines the fundamental relationships between physical stimuli and their cognitive-behavioral reactions. One family of widely studied tasks in this domain is that of alternative forced choice (AFC) tasks. Not unlike the tasks presented in the previous chapters of this work, subjects are oft asked to make actions in which some outcomes lead to reward. For example, in a simple sensory discrimination task, a subject may be asked wether they see blue color on a screen. Plotting responses against the intensity of blue hue generates a so called psychometric curve. In it, we would see that the proportion of correct detections would increase with hue. This observation allows us to draw a direct relationship between the task and behaviour. With complex tasks, this relationship is much harder to establish. Moving a rook or a knight in a game of chess is not easily relatable to any task parameter.

The dichotomy presented has led to a great divide in the complexity of tasks studied by the fields of AI and of Cognitive Sciences. A way to quantify complexity in a task is by observing the dimensions of its state-space - that is, the number of possible different states that an agent can reach in any possible sequence of actions.

Decision trees - i.e. schematic representations of the state-space - are a powerful way to model planning tasks [van Opheusden and Ma, 2019]. While admittedly reductionist, as it ignores things like task difficulty and wether there is uncertainty in the task, by calculating the dimensions of a decision tree we can derive a measure of task complexity.

van Opheusden and Ji Ma (2019) compare several families of tasks as to their state-space complexities. They pose that

melanie mitchel: AI a guide for thinking humans - part 4 on the measure of intelligence - chollet filter new kids for g's lab

To bridge this gap, we have developed a novel game specifically tailored to explore and illuminate the mechanisms of human skill learning. This game, unlike traditional ones, is uniquely engineered to delve into the depths of how humans learn, adapt, and master various strategies and skills. It's not just a platform for challenging human intellect against artificial intelligence but a meticulously crafted tool to dissect and understand the underlying processes of human learning. Our game aims to provide valuable insights into the cognitive journey of skill acquisition, offering a new perspective on how humans rapidly adapt and excel in strategic environments. Through this game, we hope to shed light on the enigmatic processes that enable humans to quickly and efficiently become skilled, thereby contributing to the broader understanding of human cognition and learning strategies as explored in the field of behavioral and brain sciences.

4.1.0.4 Ethologically relevant tasks

4.1.0.5 Set and setting

4.1.0.6 Scalability

4.2 Materials and Methods

4.2.1 hexxed

To investigate how complex skill learning ensues, the right tool is needed. After exploring the landscape of possible tasks[4.1], we developed a novel task: *hexxed*. *hexxed* is a puzzle-like videogame task that challenges it's solvers to explore and solve complex problems. Solving *hexxed* requires carefully planning moves ahead in both space and time.

4.2.2 The task

While *hexxed* takes the shape of a game, it is designed first-and-foremost as a psychophysics task.

The game is interacted with by touching on a touch-sensitive device.

When first presented with it, the players are immediately shown the first puzzle with no further instructions other than the displayed on-screen: "controls: tap and swipe".

In *hexxed*, the goal is to rotate a collector (which is the player's controllable component) around lobes of a hexagon and collect rewards. Rewards are administered when targets are collected. Targets can spawn in any of the lobes of the hexagon (with a minimum of 1 target in total and a maximum of 1 target per lobe).

In *hexxed*, time is discretized. For every action a player takes, time progresses one unit into the future. This means that hexxed has no inherent time-pressure to be solved and that it is the players who determine the pace at which they solve it.

At any point, players can make one of two actions. To tap or swipe one of the lobes.

Tapping on any of the lobes causes the collector (i.e. the player) to move there. The collector only moves one lobe at a time. If the player clicks the lobe adjacent to them, they move there and progress 1 unit of time. If the player clicks a lobe that is non-adjacent to their position, time progresses the corresponding distance (e.g. a player clicks the lobe on the opposite side of the hexagon - 3-away - and time progresses 3 units). Whenever time progresses, targets grow in size. Targets of size 1, are worth the least possible reward, whereas targets of size 6 are worth maximum reward. Targets of size 7 expire and are no longer visible or valuable to the player. Swiping on a lobe without a target is treated as a tap. Swiping on a target collects points corresponding to the target's size.

Figure 1: Targets grow in size and value up until a threshold, in which they are no longer valuable. Player actions determine this growth.

The task is then to navigate the hexagon's lobes and collect targets as large as possible.

The task is designed as an np-complete problem. Np-completeness (or nondeterministic polynomial time completeness) refers to when the solution for any possible problem in a task can be verified in polynomial time - using a brute-force search (i.e. an algorithm that attempts all possible solutions). This is a crucial feature of *hexxed*, as it allows us to arbitrarily manipulate the complexity of any puzzle while always knowing the optimal solution is within computational reach. Np-completeness grows the posed challenge exponentially, giving more dynamic range for testing the limits of performance. Simultaneously, the posed problem becomes isomorphic with a large class of well-studied problems.

Complexity in hexxed can be manipulated in two ways: 1) by changing the number of targets on screen and 2) by changing the order in which targets are spawned. A growing number of targets forces the player to move more around the hexagon to collect them, whereas the order in which targets are spawned influences how those movements are made.

Figure 2: Targets grow in size and value up until a threshold, in which they are no longer valuable. Player actions determine this growth.

The task is comprised of 6 different levels of difficulty - corresponding to the number of targets displayed on screen for that level. In the first 4 levels, the players are probed with all the possible puzzles (combinations of spawn locations and their rotational variants). In levels 5 and 6, only half the possible puzzles are shown. To progress in the task, players must approximate optimality to a varying degree (e.g. for level 1, players can lose no more than 15 points in order to complete the level).

4.2.3 Scoring

Individual targets grow up to six times and are worth n^2 points: from 1^2 - as they spawn - to 6^2 points - at their ripest. The exponential nature of target value places weight on collecting the target as large as possible. However - given that the game is self-paced - too many interactions with the screen will exhaust a target and lead to 0 reward (see 4.2.2 for the task design).

4.2.4 Action-sequences

For analysis purposes, we wanted to understand what serial combinations of actions - or action sequences - the players used to solve hexxed. To do so, we number the 6 lobes of the hexagon from 0 to 5 clockwise. We number the lobe where the first target in a puzzle appears as 0. We then encode the sequence of touches or swipes (the two valid actions) on these lobes as a vector of actions. Touches are encoded as the number of the lobe that was pressed l and swipes are encoded as the negative of said lobe number -l.

In an example action sequence for level 1, a subject performed a tap on the lobe to the left of the lobe which had a target; then, the player touched on the lobe with the target for 5 times; finally, the player swiped on the lobe with the target to get maximal reward. This action sequence is encoded as [5,0,0,0,0,0,-0].

4.2.5 Optimality in hexxed

Due to *hexxed's* design, we are capable of calculating the optimal solution for any given puzzle.

To calculate the optimal course of actions, we trace which action-sequences lead to the maximum possible score for a given puzzle (see figure 4).

Because of the nature of the game design, in some circumstances, several action-sequences may be equally optimal. As the puzzle complexity increases, however, the fraction of optimal action-sequences becomes smaller.

For level 1, optimality is trivial since a single target can always be collected at maximum value: Any combination of moves that ends at the target lobe after 6 actions allows for maximal point collection.

For level 2, two targets are available for collection. However, depending on wether targets are in adjacent lobes or not, they may or not be both collectible at maximum size (see figure 2). This means that, on non-adjacent targets, the player must sacrifice points of the first target to collect the second at its ripest. When targets are n lobes away from each other, the maximum possible score is to collect the first target at step $6 - n + 1$ so that there are enough moves available to rotate n lobes and collect the second target at its maximum value (e.g. for two targets that are 3 lobes away from each other, the maximum ripeness is of $(6 - 2) + 6 = 10$ or $(6 - 2)^2 + 6^2 = 52$ in points.

For levels 2 through 6, two competing heuristics can be used to maximize reward efficiently: targets can be collected a) in order of their appearance or b) in order of their proximity (see figure 2). As puzzle complexity increases to levels 3 and onward, collecting targets in their order of appearance is no longer the optimal strategy.

The way hexxed is designed, thus, forces players to constantly rethink and revisit their previously optimal policies. By virtue of its NP-completeness, hexxed requires players to update their policies to incorporate progressively higher-order dependencies, making it a great candidate to investigate complex skill learning.

4.2.6 The game

hexxed is instantiated a game. Unlike typical lab-based research tasks, *hexxed* was developed using a game engine. Game engines are a family of software that allows for quick and precise manipulation of both algorithmic logic and graphics. *hexxed* was developed using *Godot* [Contributors, 2023], an open-source game engine chosen for its open-ness and flexibility.

In the process of coding *hexxed*, we leveraged its algorithmic nature and made it entirely generative - The game does not consist of static images and illustrations, instead, the graphics are generated on the device's graphics card. This allows for an extremely small executable size: the entire game is only ∼3.5mb in size. The game is available on both android and iOS devices and, due to its small size and speed, the game is available on an estimated 95% of all smart devices.

All of *hexxed*'s code is open-source and available under a Creative Commons license at `https://hexxed.io`.

4.2.7 Data collection and recruitment

The data collection process for the *hexxed* experiment encompassed both laboratory and online environments.

In the controlled laboratory setting, study participants engaged with the game on a large touch-sensitive screen located within a quiet, controlled environment. The participant pool consisted of individuals from the Champalimaud Research community, comprising undergraduate students, technicians, graduate students, and postgraduates. It is important to note that unless otherwise specified, the *hexxed* dataset presented in this dissertation primarily stems from online data collection.

Conducting data collection online introduced unique challenges, notably those related to scalability and the breadth of data collection. By framing the experiment as a game, we anticipated data to be generated at varying and unpredictable rates, as the game was accessible to anyone, anywhere, at any time. To maximize participation, we collaborated with the science communication offices of both Champalimaud Research and the University of California, Berkeley, resulting in extensive media coverage that attracted a substantial number of participants. Consequently, we had to develop tailored solutions for data formatting, storage, and transfer to effectively manage the influx of data.

It is essential to emphasize that, despite its gamified presentation, *hexxed* remains a scientific experiment. Therefore, we diligently adhered to the ethical and deontological guidelines set forth by the Champalimaud Research ethics committee. This encompassed ensuring compliance with the General Data Protection Regulation (GDPR), obtaining informed consent from participants, ensuring data anonymization, and providing participants with the option to opt out, among other ethical considerations. The comprehensive license agreement and privacy policy governing data usage for *hexxed* can be accessed at the following URL: `https://hexxed.io`. Additionally, any ethical approval documentation and related ethics materials can be made available upon request through the same URL: `https://hexxed.io`.

4.2.8 Data Architecture

To manage the data generated by *hexxed*, we developed a customized data format implemented within a JSON structure. The data handling process involves the *hexxed* executable, which generates a file that logs every interaction players have with the screen during a playthrough. Each interaction, encompassing actions such as touches or swipes, results in the creation of a list containing pertinent data related to that specific interaction. These lists are dynamically generated and continually appended to memory as the game progresses.

At the culmination of a playthrough, which occurs either when a player completes or fails a level or when they exit the game, all the accumulated lists stored in memory are encoded into a JSON payload. This payload is subsequently transmitted to a secure server designated for this purpose.

Upon receipt at the server-side, the incoming payload is archived within an SQL database specifically designed to store data from all playthroughs across all players. This database serves as a comprehensive repository containing data from all facets of the game.

For a detailed illustration and documentation of the payload structure, an example is provided in Supplementary Table 5. This table offers insight into the format and contents of the JSON payload, aiding in the comprehension and analysis of the data collected during *hexxed* playthroughs.

4.2.9 Data Processing and Analysis

Due to the size and complexity of the collected data, we had to build a custom pipeline for processing and analysis. All of the python pipeline scripts and documentation can be found on *hexxed's* github repository at `https://hexxed.io`.

After downloaded, the data is extracted from the JSON payload into a pandas dataframe. This process requires careful consideration, as it is easy to underestimate the needed computational resources when dealing with such a large dataset.

Raw data is saved in a compressed format. Now, its efficient parsing becomes the challenge. Because of the payload structure, to extract the data into an analysis-friendly format we naively would have to iterate through thousands of rows, parsing each JSON string as appropriately *typed* columns in a dataframe. Luckily, python and pandas allow for alternatives to this iterative process. Using the split-apply-combine [Wickham, 2011] framework, and after thorough optimization steps, we transform our entire dataset into a large long-formatted pandas dataframe (i.e. a timeseries). Following the preprocessing steps, the data is now easily *query-able* and manipulable. Our final dataframe has a coarse-ness of one action per row - the highest granularity the raw data allows

for. Using pandas groupby, it is easy to adjust on which variable we want to group this dataframe, allowing for efficient analysis of any type. All of these steps took our data processing pipeline from approximately two hours to no more than two minutes.

Table 1: Table Title for Part 1

device-ID	session-start	time	sw-flip	sw-offset	sw-subwave-num
0	0012e848890bbcb5	522282	531464	False	4.000000
1	0012e848890bbcb5	522282	533139	False	4.000000
2	0012e848890bbcb5	522282	534881	False	4.000000
3	0012e848890bbcb5	522282	540839	False	4.000000
4	0012e848890bbcb5	522282	542457	False	4.000000

sw-number	mo-y	mo-x	mo-fake-release	mo-act-drag	mo-lobe
2.000000	1.000000	-121.111099	196.539368	0.000000	False
2.000000	1.000000	82.777802	-131.273987	0.000000	False
2.000000	1.000000	-87.222214	103.195862	0.000000	False
5.000000	2.000000	127.222229	202.651123	0.000000	False
5.000000	2.000000	246.111145	-10.149719	0.000000	False

level	version	device-timezone	device-OS	time-diff	coord-diff
5.000000	1.000000	'bias': 0.0, 'name': 'WET'	Android	-49938.000000	318.914062
2.000000	1.000000	'bias': 0.0, 'name': 'WET'	Android	1675.000000	-531.702271
5.000000	1.000000	'bias': 0.0, 'name': 'WET'	Android	1742.000000	404.469849
6.000000	1.000000	'bias': 0.0, 'name': 'WET'	Android	5958.000000	-114.989182
1.000000	1.000000	'bias': 0.0, 'name': 'WET'	Android	1618.000000	-331.689758

While shortly described, we found this process to be extremely valuable. As the scope and dimensionality of datasets increases in behavioral sciences, it is increasingly important that scientists adopt novel strategies with which to handle their data. Efficient data handling and transparency allow for replicability and 3rd party analysis/criticism/expansion, and thus, all our processing and analysis scripts are available at `https://hexxed.io`.

4.2.10 Artificial agents solving hexxed

As part of our research and leveraging the careful task design, we were able to ask artificial agents to solve our task. To facilitate this, a collaborator of our team encoded the task within a GYM environment. GYM is a library that offers a standardized API for communication between reinforcement learning agents and a task of interest. Several agents were exposed and trained in our task: *1 Layer Linear, 3-Layer ReLU, 1 Conv. Layer + 3-Layer ReLU*) Performance from all agents was similar. With more complexity in agent design, we found increases in the rate at which the task could be learned. Marginally, the agent that learned the fastest was the *1 Convolution Layer + 3-Layer ReLU*, thus, the results shown in this work will be from this agent.

4.2.11 Beyond this thesis

currently in bullet points only

- The specific implementations of the agents
- All levels above 1
- Lab collected data

4.3 Results

add the agents plots and text/narrative

4.3.1 Introduction

hexxed was designed to explore complexity in learning and, as such, learning comprises a large part of this work. In fact, the scope of this thesis will solely be on the phase where subjects learn the rules of the game - level 1.

explain the role of the plots of the agents

4.3.2 "Picky" - Humans explore the space of possible rewards unevenly

hexxed's dataset is vast. Let us start with a low level observation: the first ever interaction with the task.

As the game provides the player with little to no instructions (see 4.2.2), how and where the players interact with the screen can give insight on whether, in broad strokes, the game is intuitive or, at the very least, interpretable.

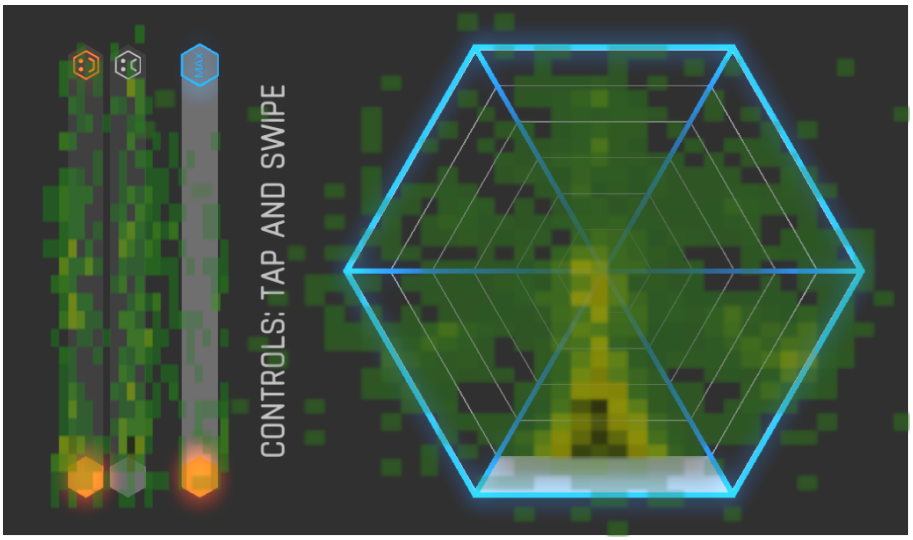

Figure 3: "First touch" - A heat-map of the participants' first ever touch coordinates on the screen, superimposed with a screenshot of the game. Player touches are largely centered around the gray bar on the bottom. Unbeknownst to the player, this bar is an indicator that a target will appear on that lobe.

In Figure 3, the first ever interaction with the game is shown. Players seem to have a common *prior* that they ought to press the hexagon on the lobe that is highlighted. That lobe is where a target becomes available for collection, thus, pressing it is a sensible option.

Due to the sheer dimension of the dataset, we will focus on the first level of the task for the rest of this work. In level 1, only one target is available for collection (see 4.2.2).

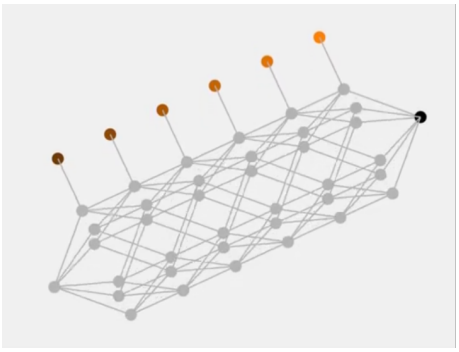

Figure 4: A visualization of level 1 in the shape of a Markov Decision Problem. From the initial node, each player action propagates them 1 step rightwards. Lines show possible transitions. The orange colored circles denote reward availability and magnitude (light hue = larger reward). The black circle symbolizes a reward of 0.

Our task can be formalized as a Markov Decision Problem (MDP). In Figure 4, we show an example visualization of said MDP for puzzles in level 1.

As players explore level 1, they eventually collect points. In Figure 5, we show the probability of our subjects, on a given trial, getting any of the seven possible reward sizes. Rewards of 0 are the most common - this reflects that failures are the most common outcome in the task. Looking at the remaining reward sizes, the second most common collected reward is the maximum reward - a requirement for optimal level completion. The only other reward size that is differently explored is the smallest reward possible.

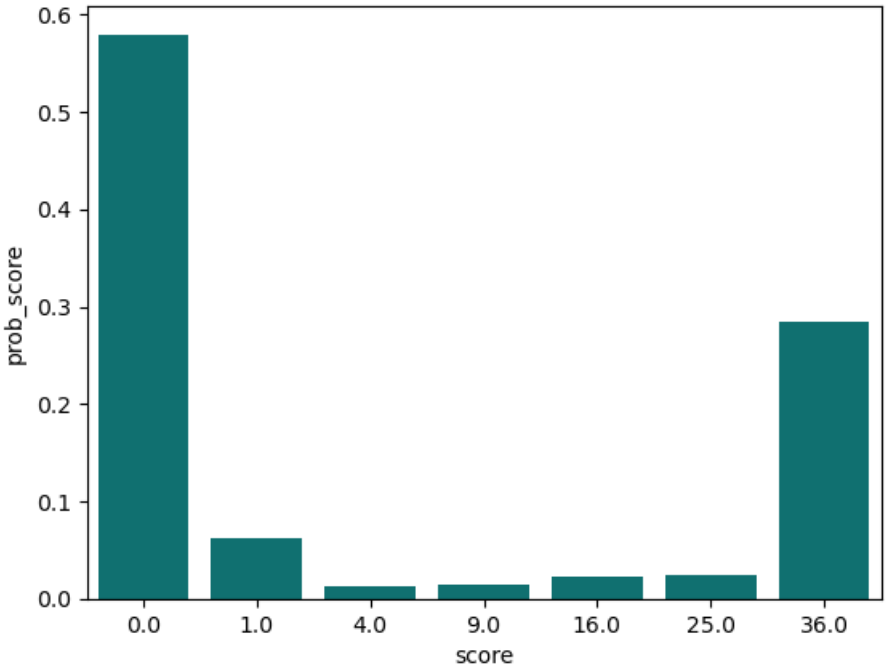

Figure 5: The probability of getting rewards for all possible reward sizes.

As continuous choices are made, specific strategies or ways of playing emerge. In Figure 6, a flattened version of the MDP of Figure 4 is populated with the action-transitions made by our subjects. The path closest to rewards is overrepresented and. As seen in Figure 5, states that lead to rewards of 0, 1 and optimal are the most commonly visited.

Our subjects explore the space of possible rewards unevenly.

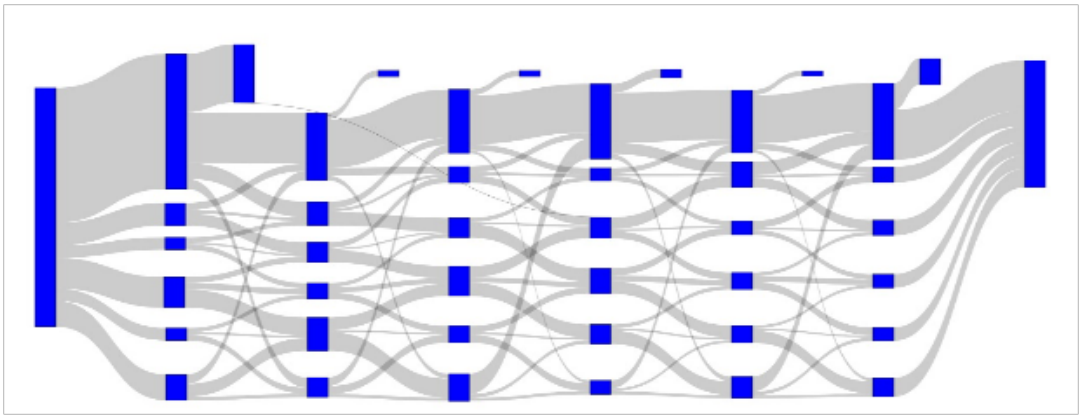

Figure 6: Action transitions in level 1 puzzles in a flattened version of 4. All subjects start on the leftmost node and traverse the plot rightwards. From the initial state, line widths represent the probability of transitions to subsequent nodes and node widths, their occupancy. Rewards are available on the nodes on the top row, growing in value from left to right. The rightmost node represents a reward of 0.

4.3.3 "Sticky" - Humans are persistent in their learning strategies

After looking at the landscape of action-transitions in a trial/puzzle, we now investigate what are the dynamics across trials. In Figure 7, we plot the conditional probability of getting reward r in trial t given the reward in the previous trial *t-1*. We observe that the edges of the diagonal seem overrepresented. This is consistent with the points raised on 4.3.2. Rewards of 0 (i.e. failures) lead to revisiting failures, whereas optimal rewards lead to optimal rewards once more. The center of the heat-map is more fuzzy. Intermediate rewards suggest intermediate sub-optimal strategies where perhaps subjects have already learned the basic controls of the game, however, are still not capable of solving it optimally.

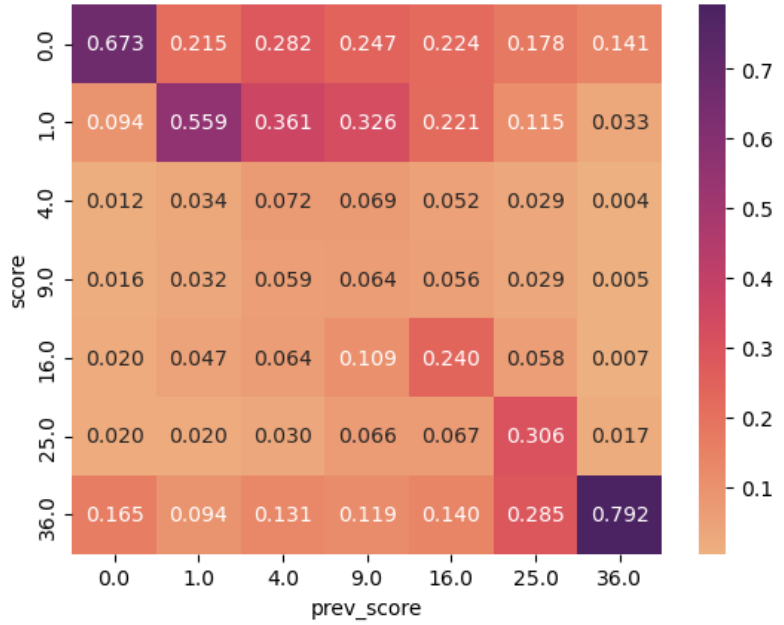

Figure 7: The probability of getting a reward r on trial t conditional on the reward of the previous trial *t-1*: $P(r_t|r_{t-1})$. Darker colors indicate higher probability. Normalized by reward size (columns).

In order to progress in the game, sequences of actions are made. Our players performed 14158 unique action sequences. In Table 4, the 20 most common are shown - the encoding is explained on section 4.2.4.

We see that the most common action is a single swipe on the correct lobe - which leads to a reward of 1. The second most common is one of the optimal solutions - tapping only the lobe where reward is available and swiping at the last possible moment. The third does not include swipes, and thus leads to a reward of 0. The fifth and seventh represent, respectively, navigating the hexagon clockwise and counterclockwise before swiping at the last possible step - both leading to optimal reward.

Now it is possible to look at the action sequence transitions from sequence s on trial t from the sequence done on the previous trial *t-1*. Again, the diagonal appears to be overrepresented. This signifies that players often repeat the same action sequence they performed before. This is especially the case for optimal solutions (e.g. sequences 2, 5 and 7), where it makes most sense to keep repeating ones current action sequence. Another feature is the asymmetry between the lower and upper sides of the diagonal. Action sequences are most commonly taken from least common to most common, perhaps signifying some conversion towards learning.

Our subjects are persistent in which actions they perform, both when looking at which reward size they explore throughout learning as well as when looking at specific action sequences.

Action_Sequence	Frequency	Rank
[-0.0]	9805	1.0
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, -0.0]	7507	2.0
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]	2978	3.0
[0.0, -0.0]	2752	4.0
[1.0, 2.0, 3.0, 4.0, 5.0, 0.0, -0.0]	819	5.0
[3.0, 0.0, 3.0]	582	6.0
[5.0, 4.0, 3.0, 2.0, 1.0, 0.0, -0.0]	535	7.0
[0.0, 1.0, 2.0, 3.0, 4.0, 5.0, 0.0]	460	8.0
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]	439	9.0
[0.0, 3.0, 0.0]	438	10.0
[0.0, 0.0, 0.0, 0.0, 0.0, -0.0]	437	11.0
[-3.0, -0.0]	406	12.0
[0.0, 0.0, -0.0]	406	12.0
[1.0, 2.0, 3.0, 4.0, 5.0, 0.0, 0.0]	318	13.0
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 1.0]	302	14.0
[0.0, 5.0, 4.0, 3.0, 2.0, 1.0, 0.0]	301	15.0
[3.0, -0.0]	279	16.0
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 3.0]	273	17.0
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0]	269	18.0
[3.0, 3.0, 3.0, 3.0, 3.0, 3.0, 3.0]	263	19.0
[1.0, 2.0, 3.0, 4.0, 5.0, 0.0, 1.0]	254	20.0

Table 4: The top 20 action sequences and respective frequencies of occurrence - in total occurrences across all players. There are a total of 14158 unique action sequences.

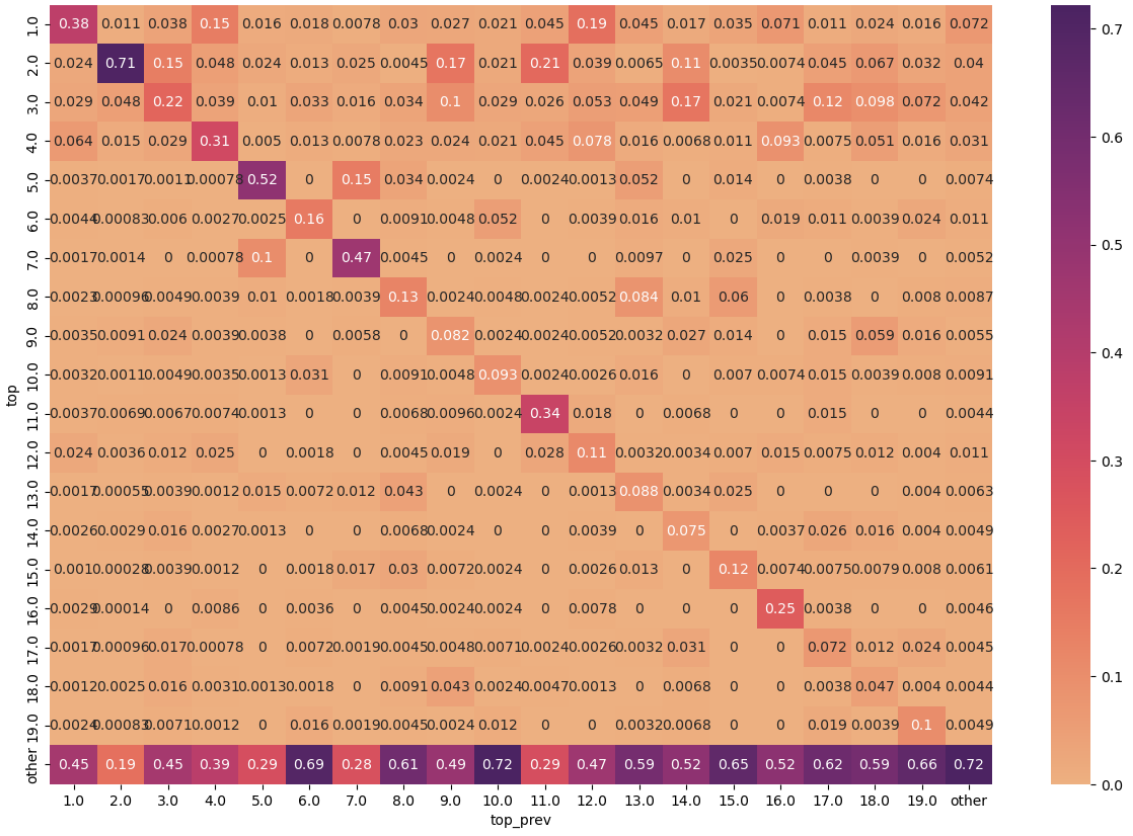

Figure 8: The probability of performing the action sequence s on trial t conditional on the sequence done on the previous trial *t-1*: $P(s_t|s_{t-1})$. The top 19 sequences are displayed and the remaining thousands are grouped as "other". Sequences are ordered by their frequency of occurrence in the data. Darker colors indicate higher probability. Normalized by sequence (columns).

4.3.4 "Leapy"

We finally zoom-out and observe the entire progression of a player throughout a level. In Figure 9, each individual player is represented by a randomly colored line. To complete level 1, subjects must collect a certain amount of reward (see 4.2.2). Here, this threshold is represented by the dashed line. In these traces we observe a pattern of low performance on a number of sessions followed by a rapid leap that leads to success. In the top panel, a histogram of when players reached the reward threshold. We see an early centered yet long tailed distribution that contrasts with the agents'.

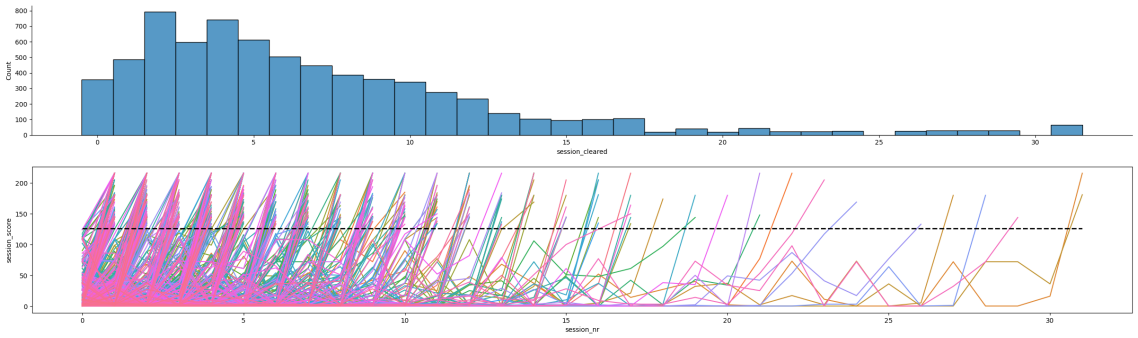

Figure 9: Bottom panel: Raw traces of reward progression across level attempts during learning (i.e. until the level is complete for the first time). In order to complete a level, subjects must reach a fixed reward threshold, here represented as a dashed line - see 4.2.2. Hue is randomly assigned for easier visual discrimination across individual traces. Top panel: Summary visualization of the bottom panel. A histogram of how many players reached/crossed the reward threshold across levels.

4.4 Discussion

4.5 Author Contributions

4.6 Acknowledgments

5 General Discussion

5.1 Brief summary of the main findings

5.2 Opportunities for expansion

5.3 Concluding remarks

6 Supplementary

Variable Name	Size	Description	Example
drone-play	(1)	time at which the background drone music restarts the loop	[374062]
mo-act-drag	(13)	0 for tap, 1 for drag (swipe).	[0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1]
mo-fake-release	(13)	??? multiple releases by a single press as registered by the device.	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
mo-lobe	(13)	lobe number (1-indexed) that was either touched or swiped, value is 0 if user touches/swipes outside the hexagon	[2, 2, 2, 2, 2, 2, 4, 4, 4, 4, 4, 4, 4]
mo-press-time	(13)	time of user action relative to start of level (milliseconds)	[365039, 365192, 365338, 365486, 365635, 366691, 368974, 369089, 369218, 369360, 369496, 369641, 371266]
mo-press-x	(13)	where you press relative to the center of the big hexagon	[-96.825806, -101.270752, -87.935974, -83.491028, -90.158447, -168.500244, 7.074341, 22.075989, 19.297913, 22.631592, 20.409119, 19.853516, 0.406982]
mo-press-y	(13)	where you press relative to the center of the big hexagon	[82.777802, 80.000031, 61.666687, 62.222229, 67.777802, 120.555573, -155, -156.666656, -155, -158.333328, -167.222214, -171.666656, -195]
mo-time	(13)	when you release	[365058, 365225, 365387, 365525, 365671, 366832, 368991, 369122, 369272, 369396, 369545, 369694, 371364]
mo-x	(13)	where you release	[-97.381409, -101.270752, -88.491577, -84.046631, -91.269653, 28.743347, 7.074341, 22.075989, 20.409119, 23.187195, 20.964722, 20.409119, -9.594116]
mo-y	(13)	where you release	[83.333344, 80.000031, 62.777802, 62.777802, 68.333344, 21.666687, -155, -156.666656, -155, -158.333328, -166.666656, -171.111099, -62.222214]
mo-act-taken-act	(16)	actions from the vantage point of slider, +/-1 if slide is moving (clockwise/counterclockwise), 2 if slider is collecting, 0 if it's aggregating (tap in place). The other numbers in the example above can actually be anything, including 0, -1, 1, and 2. In order to disambiguate these, look at mo-act-taken-pos, the entries that are -1 correspond to these random numbers here that indicate begin/end of a puzzle.	[50, 0, 0, 0, 0, 0, 2, -1962528752, 0, 0, 0, 0, 0, 0, 2, -1962528752]

Variable Name	Size	Description	Example
mo-act-taken-pos	(16)	Each -1 "bookends" the start/end of a puzzle, other numbers refer to the lobes (0-indexed)	[-1, 1, 1, 1, 1, 1, 1, -1, 3, 3, 3, 3, 3, 3, 3, -1]
mo-act-taken-time	(17)	godot signal system logs these "taken" values into a queue as soon as the slider movement is done (tween completion), e.g.,	[364621, 365338, 365652, 365952, 366254, 366565, 367116, 368208, 369289, 369598, 369905, 370218, 370532, 370835, 371668, 371976, 374062]
mo-move-pos-x	(24)	every time Android sends a drag (swipe) signal, it will write to these arrays. These include taps that are misunderstood as drags/swipes by Android, but Godot logic has a more stringent requirement to count an interaction as a swipe.	[-97.381409, -88.491577, -88.491577, -84.046631, -91.269653, -91.269653, -167.944641, -166.277771, -161.277283, -152.38739, -133.49646, -101.270752, 28.743347, 19.853516, 20.409119, 23.187195, 20.964722, 20.409119, 0.406982, -0.148682, -1.259888, -4.037964, -9.038513, -9.594116]
mo-move-pos-y	(24)		[83.333344, 62.222229, 62.777802, 62.777802, 67.777802, 68.333344, 120.000031, 118.888916, 117.222229, 112.777802, 102.777802, 85.555573, 21.666687, -155, -155, -158.333328, -166.666656, -171.111099, -194.444443, -192.222214, -182.222214, -160, -76.111099, -62.222214]
mo-move-time	(24)		[365057, 365371, 365387, 365525, 365670, 365670, 366725, 366743, 366761, 366778, 366796, 366814, 366832, 369254, 369271, 369395, 369529, 369694, 371282, 371298, 371315, 371331, 371363, 371364]
ba-ID	(2)	The size rank of the object collected, biggest target=0, second biggest=1, and so on.	[0, 0]
ba-age	(2)	size at which the object was collected, if you miss it, it's indicated by a 7 – the value is at least 1.	[5, 6]
ba-position	(2)	the lobe that the collected object was on (and by definition was collected from)	[1, 3]
ba-time	(2)	time of collection	[367116, 371651]
focus-in	(2)	when the player switches back into the app	[379081, 386834]
focus-out	(2)	when the player switches out of the app	[374954, 381247]
sw-time-end	(2)	time that subwave (puzzle) ended.	[367116, 371651]
sw-flip	(3)	whether flipped or not (after rotational offset) around the first piece.	[0, 1, 0]
sw-offset	(3)	rotational offset of objects (0-indexed)	[1, 3, 4]

Variable Name	Size	Description	Example
sw-subwave-num	(3)	the puzzle id according to the spawn.txt (see section on spawn.txt below)	[0, 1, 5]
sw-time	(3)	start time of the subwave (puzzle)	[364339, 367914, 373764]
device-IP	(7)		['192.168.1.66', '2001:569:7ac6:6900:1457:626f:c220:f868', '2001:569:7ac6:6900:ee9b:f3ff:fe96:3eb5', 'fe80:0:0:0:ee9b:f3ff:fe96:3eb5', 'fe80:0:0:0:ec9b:f3ff:fe96:3eb5', '127.0.0.1', '0:0:0:0:0:0:0:1']
level	(1)	current level NOT 0-indexed.	1
version	(1)		1.08
rand-pos	(1)	a/b testing: which category you were put into...	2
device-ID	(1)	unique identifier of the physical device of the user (did not used to be accessible by older Godot versions)	a3f1ef82e68aa9bb
device-OS	(1)		Android
level-won	(1)	only True if the session is saved upon successful completion of the level. NOTE: if False, it does not mean that this payload was sent upon the user LOSING the level. Since they could have focused out: if focus-out is the last event of the game then that's a good indication for this scenario. The only way you can judge loss of level is if the total number of points earned falls below the threshold of points needed for successful completion	False
user-quit	(1)	if the user pressed back. It does NOT register a quit if the user focuses OUT and then closes the app.	False
device-dpi	(1)		420
repeat-bad	(1)	a/b testing variable: whether you present the same pattern if a person fails it... currently disabled i.e. they don't see the same puzzle again.	2
device-name	(1)		SM-N920W8
total-saves	(1)	number of times this user's files have been saved: cumulative across all levels; incremented for each focus-out event.	13
max-unlocked	(1)	maximum level unlocked PRIOR to this attempt, 0-indexed	1
OS-start-time	(1)		364316

Variable Name	Size	Description	Example
device-ID-rand	(1)	random number assigned to the user the first time they open the game after install used in conjunction with below in lieu of device-ID in older Godot versions	2846276096
device-ID-time	(1)	time at which the user opens the game for the first time after install: used in conjunction with above in lieu of device-ID in older Godot versions	1592595456
failure-thresh	(1)	level dependent, always 15 except last level, which is set to 5.	15
device-timezone	(1)		'bias': -420, 'name': 'PDT'
device-kb-locale	(1)		en-CA
device-current-time	(1)		'day': 22, 'dst': True, 'hour': 14, 'year': 2020, 'month': 6, 'minute': 51, 'second': 40, 'weekday': 1
device-screensize-x	(1)		1920
device-screensize-y	(1)		1080
device-video-driver	(1)		0

Table 5: An example of *hexxed's* data structure as sent by the remote device in four columns: Column 1 - The variable name; Column 2 - How many times this variable was logged for this given example; Column 3 - Variable description; Column 4 - An example value for the variable

References

[Contributors, 2023] Contributors, G. E. (2014–2023). Godot engine. `https://godotengine.org/`.

[Hsu, 2002] Hsu, F.-h. (2002). *Behind Deep Blue: Building the Computer that Defeated the World Chess Champion*. Princeton Paperbacks. Princeton University Press.

[Lake et al., 2017] Lake, B. M., Ullman, T. D., Tenenbaum, J. B., and Gershman, S. J. (2017). Building machines that learn and think like people. *Behavioral and brain sciences*, 40:e253.

[Martin-Maroto and de Polavieja, 2018] Martin-Maroto, F. and de Polavieja, G. G. (2018). Algebraic machine learning. *CoRR*, abs/1803.05252.

[Mnih et al., 2015] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A. A., Veness, J., Bellemare, M. G., Graves, A., Riedmiller, M., Fidjeland, A. K., Ostrovski, G., et al. (2015). Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533.

[Tsividis et al., 2017] Tsividis, P. A., Pouncy, T., Xu, J. L., Tenenbaum, J. B., and Gershman, S. J. (2017). Human learning in atari. In *2017 AAAI spring symposium series*.

[van Opheusden and Ma, 2019] van Opheusden, B. and Ma, W. J. (2019). Tasks for aligning human and machine planning. *Current Opinion in Behavioral Sciences*, 29:127–133.

[Wickham, 2011] Wickham, H. (2011). The split-apply-combine strategy for data analysis. *Journal of statistical software*, 40:1–29.